File Edit View Settings Help

```python


class Robot(object):
    ''' The Robot class, which includes the vehicles current model of the world and IPP algorithms.'''

    def __init__(self, **kwargs):
        ''' Initialize the robot class with a GP model, initial location, path sets, and prior dataset
        Inputs:
            sample_world (method) a function handle that takes a set of locations as input and returns a set of observations
            start_loc (tuple of floats) the location of the robot initially in 2-D space e.g. (0.0, 0.0, 0.0)
            extent (tuple of floats): a tuple representing the max/min of 2D rectangular domain i.e. (-10, 10, -50, 50)
            kernel_file (string) a filename specifying the location of the stored kernel values
            kernel_dataset (tuple of nparrays) a tuple (xvals, zvals), where xvals is a Npoint x 2 nparray of type float and zvals is a Npoint x 1 nparray of type float
            prior_dataset (tuple of nparrays) a tuple (xvals, zvals), where xvals is a Npoint x 2 nparray of type float and zvals is a Npoint x 1 nparray of type float
            init_lengthscale (float) lengthscale param of kernel
            init_variance (float) variance param of kernel
            noise (float) the sensor noise parameter of kernel
            path_generator (string): one of default, dubins, or equal_dubins. Robot path parameterization.
            frontier_size (int): the number of paths in the generated path set
            horizon_length (float): the length of the paths generated by the robot
            turning_radius (float): the turning radius (in units of distance) of the robot
            sample_set (float): the step size (in units of distance) between sequential samples on a trajectory
            evaluation (Evaluation object): an evaluation object for performance metric compuation
            f_rew (string): the reward function. One of {hotspot_info, mean, info_gain, exp_info, mes}
                    create_animation (boolean): save the generate world model and trajectory to file at each timestep
        '''

        # Parameterization for the robot
        self.ranges = kwargs['extent']
        self.dimension = kwargs['dimension']
        self.create_animation = kwargs['create_animation']
        self.eval = kwargs['evaluation']
```